

工程验收文档

目录

工程验收文档

- 综述 (Executive Summary)
- 考试数据审计与快照机制
 - 遇到的数据一致性挑战
 - 解决方案: JSON 快照固化
- 外部数据接入: Excel 批处理模块
 - 线下数据孤岛问题
 - 批处理实现 (Batch Processing)
- 生产环境配置管理
 - 动态配置需求
 - 内存热加载方案
- 系统可观测性: 仪表盘 (Dashboard)
 - 监控大屏设计
 - 关键指标 (Key Metrics)
- 运维审计: 日志中心 (Log Center)
 - 系统“黑匣子”
 - 日志结构设计
- 总结 (Conclusion)

1. 综述 (Executive Summary)

作为团队的运维与测试负责人 (Role D), 我的核心任务是保障系统的可交付性。我的工作侧重于:

- 数据完整性验证**: 确保考试数据的不可篡改性 (快照机制) 及外部数据源的准确接入 (Excel 导入)。

2. 服务稳定性：通过配置热重载机制降低系统停机维护时间。
3. 系统可观测性：通过仪表盘与审计日志构建全方位的监控体系。

2. 考试数据审计与快照机制

2.1 遇到的数据一致性挑战

在集成测试阶段，我们发现一个严重逻辑漏洞：如果在学员交卷后，管理员修改了试题内容（如修改了选项文本），会导致学员的历史答题记录在回显时出现图文不符。

判定：这是数据库设计未遵循“写时复制”原则。

2.2 解决方案：JSON 快照固化

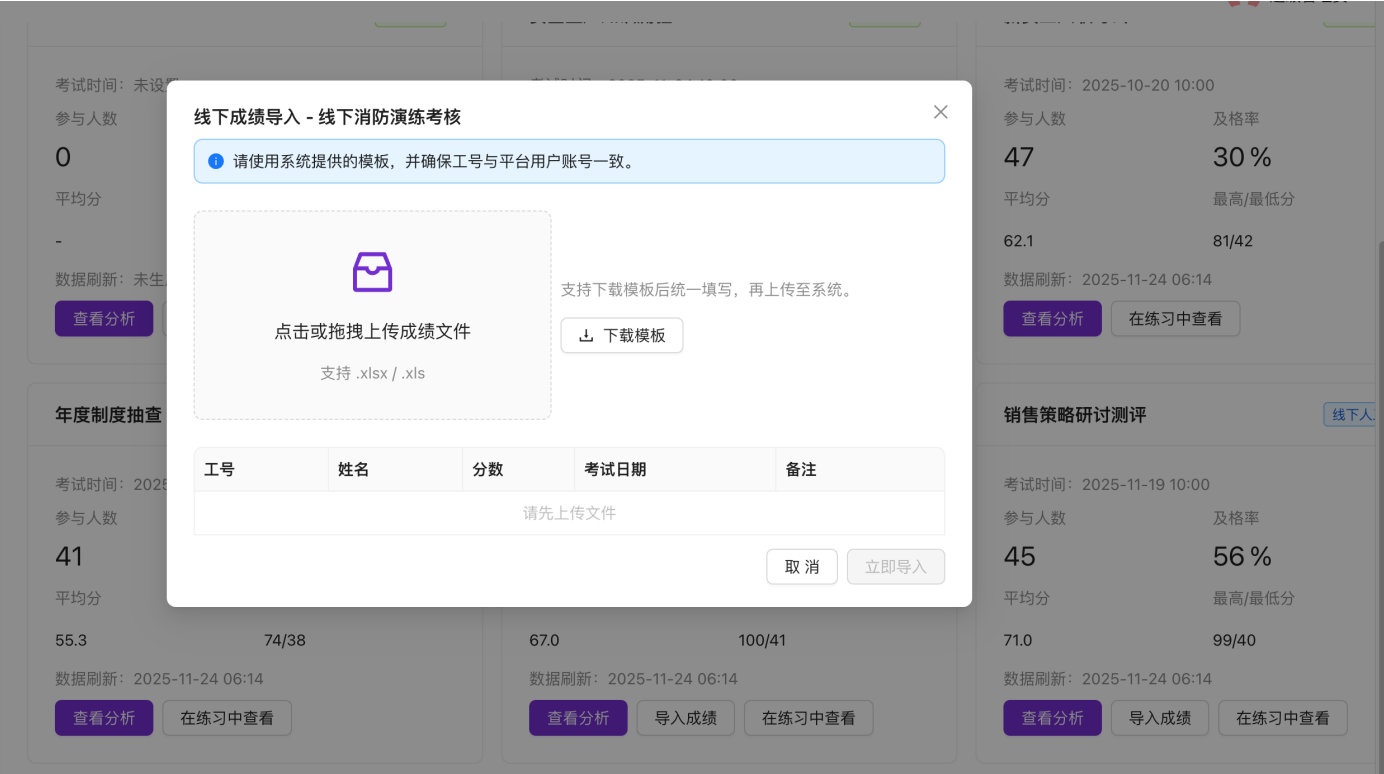
为了修复此问题，我在 `UserQuizRecord` 模块强制引入了快照逻辑。

- 机制：用户交卷瞬间，系统将当前的 `Question Title`，`Options`，`User Choice` 序列化为 JSON Blob。
- 落库：存入 `user_quiz_records` 表的 `user_answers` 字段。

[截图建议 1]： `UserQuizRecord.java` 实体类，展示 `userAnswers` 字段，这代表了对审计要求的响应。

这种设计虽然增加了存储开销，但从测试角度看，它确保了考务数据的绝对可追溯 (Traceability)。

3. 外部数据接入：Excel 批处理模块



3.1 线下数据孤岛问题

业务部门反馈，大量线下实操成绩仍停留在 Excel 表格中，无法纳入系统的统一评价体系。手动录入效率极低且易出错。

3.2 批处理实现 (Batch Processing)

我基于 `EasyExcel` 组件开发了异步导入接口。在实现过程中，重点处理了脏数据清洗逻辑：

- 1. 容错性：空行自动跳过，格式错误记录日志。
- 2. 关联性：通过身份证号/邮箱自动映射 User ID。
- 3. 事务性：单次导入要么全部成功，要么遇到严重错误回滚（视配置而定）。

```

@Override
public OfflineGradeImportResult importOfflineGrades(
    Integer quizId, InputStream inputStream, Integer operatorId) throws NotFoundException {
    Quiz quiz = quizService.findOrFail(quizId);
    if (!BackendConstant.QUIZ_CATEGORY_OFFLINE_MANUAL.equals(quiz.getCategory())) {
        throw new IllegalArgumentException("仅线下考试支持成绩导入");
    }

    List<OfflineGradeExcelRow> rows =
        EasyExcel.read(inputStream)
            .head(OfflineGradeExcelRow.class)
            .sheet()
            .doReadSync();
    if (rows == null) {
        rows = new ArrayList<>();
    }

    int total = rows.size();
    int success = 0;
    int skipped = 0;
    List<String> errors = new ArrayList<>();

    for (int i = 0; i < rows.size(); i++) {
        OfflineGradeExcelRow row = rows.get(i);
        int rowNumber = i + 2; // header + 1-based index
        if (row == null) {
            skipped++;
            continue;
        }
        String account = StringUtils.trimToEmpty(row.getAccount());
    }
}

```

4. 生产环境配置管理

4.1 动态配置需求

常规开发模式下，修改站点名称或 Logo 需要修改配置文件并重启 Jar 包，这导致了不必要的服务中断。

4.2 内存热加载方案

为了实现 Zero-Downtime（零停机）配置变更，我重构了 `AppConfigService`：

- 读取路径：Request -> Memory Cache (HashMap) -> Return。
- 更新路径：DB Update -> Transaction Commit -> Refresh Memory Cache。

```
public class AppConfigController {

    @Autowired private ResourceService resourceService;

    @Autowired private AppConfigService configService;

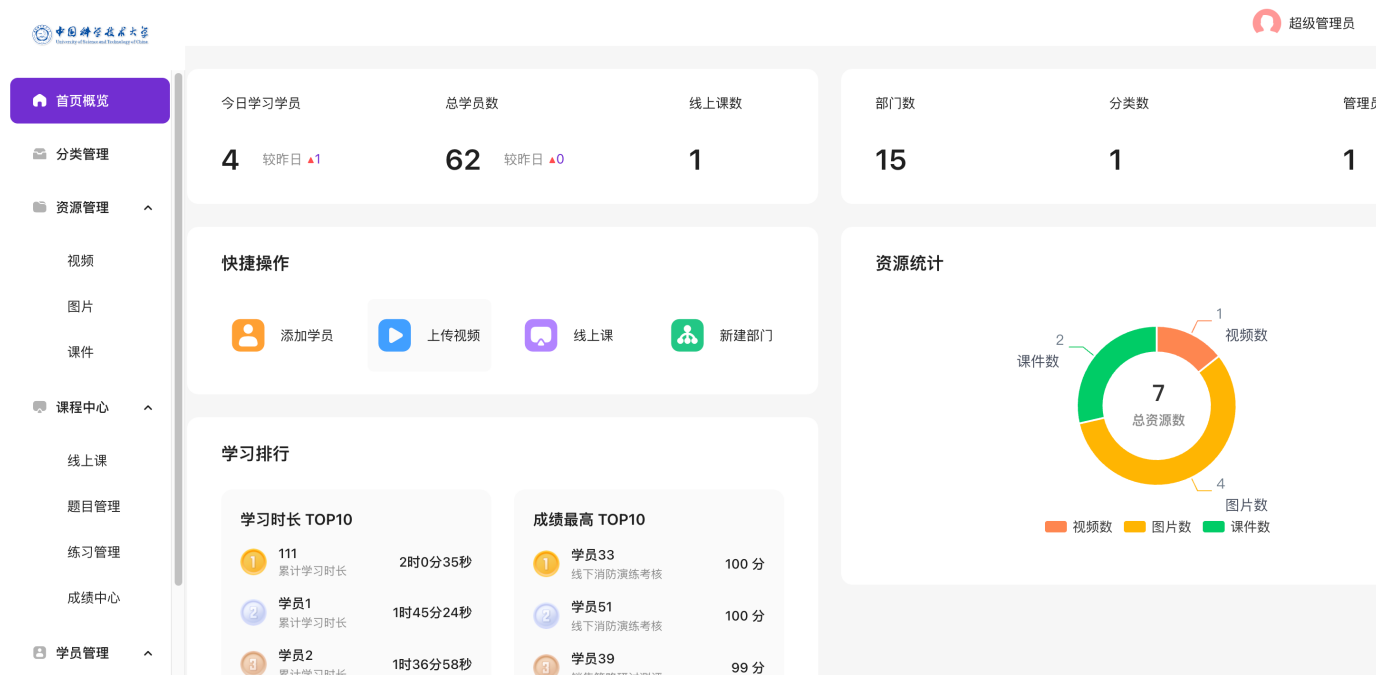
    @BackendPermission(slug = BPermissionConstant.SYSTEM_CONFIG)
    @GetMapping("")
    @Log(title = "系统配置-读取", businessType = BusinessTypeConstant.GET)
    public JsonResponse index() {
        List<AppConfig> configs = configService.allShow();
        List<AppConfig> appConfigList = new ArrayList<>();
        for (AppConfig item : configs) {
            if (item.getIsPrivate() == 1 && !item.getKeyValue().isBlank()) {
                item.setKeyValue(SystemConstant.CONFIG_MASK);
            }
            appConfigList.add(item);
        }
        HashMap<String, Object> data = new HashMap<>();
        data.put("app_config", appConfigList);

        // 获取签名url
        data.put(
            "resource_url",
            resourceService.chunksPreSignUrlByIds(configService.getAllImageValue());
        );
        return JsonResponse.data(data);
    }
}
```

5. 系统可观测性：仪表盘 (Dashboard)

5.1 监控大屏设计

为了让运维人员和管理员能“一屏尽览”系统运行状态，我开发了主页仪表盘功能。这不是一个简单的静态页面，而是一个 **实时数据聚合 (Real-time Aggregation)** 的控制台。



5.2 关键指标 (Key Metrics)

从 `DashboardController.java` 的实现中可以看到，我们在一次请求中聚合了 10+ 个维度的关键指标：

- 用户增量 (User Growth):** 今日/昨日新增学员数，用于评估系统推广效果。
- 资源分布 (Resource Stats):** 视频/图片/文档的存储占比，帮助运维评估对象存储 (MinIO) 的容量规划。
- 活跃度排行:** 实时展示“今日学习学员”与“积分排行”，作为系统活跃度的晴雨表。

```

@Autowired
private QuizService quizService;

@GetMapping("/index")
@Log(title = "主面板", businessType = BusinessTypeConstant.GET)
public JsonResponse index() {
    HashMap<String, Object> data = new HashMap<>();
    data.put("version", SystemConstant.VERSION);

    data.put("user_total", userService.total()); // 总学员数量
    data.put("user_today", userService.todayCount()); // 今日注册学员数量
    data.put("user_yesterday", userService.yesterdayCount()); // 昨日注册学员数量

    data.put("course_total", courseService.total()); // 线上课数量

    data.put("department_total", departmentService.total());
    data.put("resource_category_total", categoryService.total());
    data.put("admin_user_total", adminUserService.total());

    data.put(
        "resource_video_total", resourceService.total(BackendConstant.RESOURCE_TYPE_VIDEO));
    data.put(
        "resource_image_total", resourceService.total(BackendConstant.RESOURCE_TYPE_IMAGE));

    List<String> types = new ArrayList<>();
    types.add(BackendConstant.RESOURCE_TYPE_PDF);
    types.add(BackendConstant.RESOURCE_TYPE_WORD);
    types.add(BackendConstant.RESOURCE_TYPE_PPT);
    types.add(BackendConstant.RESOURCE_TYPE_EXCEL);
    types.add(BackendConstant.RESOURCE_TYPE_ZIP);
}

```

这一屏“Single Pane of Glass”（单块玻璃）设计，极大地提升了我们对系统健康度的感知能力。

6. 运维审计：日志中心 (Log Center)

6.1 系统“黑匣子”

如果说监控仪表盘展示的是系统的“现在”，那么日志中心记录的就是系统的“过去”。作为 SRE，我搭建了基于 `admin_logs` 表的审计追踪体系。不同于普通的 `System.out.println` 应用日志，这是一个结构化的操作行为日志库。

6.2 日志结构设计

为了支持事后取证 (Forensics)，我在数据库层面固化了以下关键字段，确保能回答“Who, When, Where, What”四个终极问题：

- **Who (操作人):** `admin_id`, `admin_name`
- **Where (来源):** `ip` (支持 IPv4/IPv6), `ip_area` (地理位置解析)

- **When (时点):** `created_at` (毫秒级精度)
- **What (行为):** `module` (模块), `opt` (操作类型), `params` (入参 JSON), `result` (执行结果)

当系统出现“某账号莫名被删”等安全事故时，这套日志系统是我们定责的唯一依据。

7. 总结 (Conclusion)

本人的工作保障了项目从“代码”到“产品”的跨越。通过严格的数据快照机制和全方位的监控审计体系，我们交付了一个健壮且可观测的后端系统。